



Roll No. : 231801096.....Sub :

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1a	13.1.26	Text Generator using N-gram Language Model		AL
1b	20.1.26	Hidden Markov Model based Predictive Text Generation		AL
2	27.1.26	Bias Audit of Pretrained Word Embedding (WEAT)		AL
3	3.2.26	GAN for Synthetic Image Generation		AL
4	10.2.26	Comparative Analysis of LLMs		AL
5	17.2.26	LLMs		AL
6	10.3.26	High Dimensionality Vector using FAISS		AL
7	17.3.26	RAG		AL
8	24.3.26	Fine tuned Model using ROUGE & BLEU		AL
Completed				
AL				

Ex no: 12
Date: 13.1.26

Text Generation using

N-gram language Model

Aim:

To implement an N-grams based predictive text model using bigram and trigram probability distributions. This helps to understand how contextual probability influences next word prediction.

PROCEDURE:

- * Initialize the environment by installing and importing the Natural Language Toolkit (NLTK).
- * Download the Brown Corpus and the universal tagset.
- * Extract raw sentence from brown corpus using `brown.sents()`
Tokenization / convert
- * All words to lowercase and flatten the nested list into a single list.
- * Build an N-Gram Model for bigram and trigram
- * Predict Next word and display Top 5 suggestions.
- * Test Bigram Prediction on certain words.

CODE:

```
from collections import defaultdict, Counter
import nltk
from nltk.corpus import brown
nltk.download('brown')
sentences = brown.sents()
```

Bigram Prediction: 'of' > 'a', 's', 'and', 'is'

Trigram prediction: 's'


```
tokens = [word.lower() for sent in sentences  
for word in sent]
```

```
print ["Tokens", len(tokens)]
```

```
def build_ngram(tokens, n):  
    model = defaultdict(counter)  
    for i in range(len(tokens) - n + 1):  
        context = tuple(tokens[i:i+n-1])  
        word = tokens[i+n-1]  
        model[context][word] += 1
```

```
    return model
```

```
bigram_model = build_ngram(tokens, 2)
```

```
trigram_model = build_ngram(tokens, 3)
```

```
print ("Models Ready")
```

```
def predict_next(context, model):
```

```
    context = tuple([w.lower() for w in  
                    context.split()])
```

```
    if context not in model:
```

```
        return []
```

```
    return [word for word in model[context].  
            most_common()]
```

```
print ("Bigram:", predict_next("Government", bigram_model))
```

```
print ("Trigram:", predict_next("Government", trigram_model))
```

RESULT:

To implement using bigram and trigram probability distribution was executed successfully.

En no 16
Date: 20-1-26

Hidden Markov Model (HMM) based Predictive Text System

AIM:

To implement a predictive text system using Hidden Markov Model, enabling students to understand contextual probability, POS transitions, and next-word prediction using Brown Corpus.

ALGORITHM:

- * Install and import NTK and download the Brown Corpus and Universal Tagset.
- * Extract sentences from the Brown Corpus using `brown.sents()`.
- * Convert all words to lowercase and flatten them into a single token list.
- * Load POS-tagged sentences using `brown.tagged_sents()` as observations and hidden states.
- * Train a Hidden Markov Model using supervised learning.
- * Compute POS transition probabilities.
- * Compute word emission probabilities.
- * Define a function to predict the next word based on POS.
- * Test the model with sample input sentences.
- * Verify grammatical correctness of the predicted words.

CODE:

```
import nltk
nltk.download("Brown")
nltk.download("universal_tagset")
```


OUTPUT:

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Unzipping corpora/brown.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data] Unzipping tagsets/universal_tagset.zip.

True

Total sentences 57340

Total tokens 1161192

['here' s

'Subtotal',

'Country' s

July 2

'said'

Grider's

HHM

wood.

570.11

RAMCO

Predictor POS: NOUN

Suggested Next

Prime

595)




```

from nltk.corpus import brown
sentences = brown.sents()
tokens = [word.lower() for sent in sentences for word
           in sent]
print("Total sentences", len(sentences))
print("Total tokens", len(tokens))
tokens[:20]

```

```

from nltk.tag import hmm
from collections import defaultdict, Counter
train_data = brown.tagged_sents(tagset='universal')
trainer = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train_supervised(train_data)
print("HMM model trained successfully.")

```

```

import math
pos_sequences = [[tag for (_, tag) in sent] for sent in
                  train_data]
pos_bigram = defaultdict(Counter)
for seq in pos_sequences:
    for i in range(len(seq)-1):
        pos_bigram[seq[i]][seq[i+1]] += 1
word_by_tag = defaultdict(Counter)
for sent in train_data:
    for word, tag in sent:
        word_by_tag[tag][word.lower()] += 1
S = 5

```

```

def predict_next_word(sentence):
    words = sentence.lower().split()
    tagged = hmm_model.tag(words)
    last_tag = tagged[-1][1]
    next_pos = pos_bigram[last_tag].most_common(1)[0][0]
    candidates = word_by_tag[next_pos].most_common(S)
    return next_pos, candidates
pos, words = predict_next_word("the government decided")
print("Predicted POS:", pos)
print("Suggested Next Words:", words)

```


Bias Audit of Pre-Trained Word

Embeddings Using the Word

Embedding Association Test

(WEAT)

En No: A

DATE: 27/11/20

AIM:

To develop a program that performs a bias audit using word Embedding Association (WEAT), to identify & quantify gender & racial stereotypes present in Google News Word2Vec embeddings.

PROCEDURE:

- * Import necessary Python Libraries: gensim, ^{numpy} numpy.
- * Load the Google News Word2Vec embeddings.
- * Define target & Attribute Word sets.
 - a. Target Words (x and y): Groups to compare.
 - i. Gender: $x = ["men", "boy", \dots]$, $y = ["women", "female"]$
 - ii. Racial bias: $x = ["European-names"]$, $y = ["African-names"]$
 - b. Attribute Words (A & B): Concepts to test association with eg.
 - i. Career vs Family $A = ["Career", "Business", \dots]$, $B = ["Family", "Home", \dots]$
- * Define Helper Functions: association function, weat score function, effect size function and compute weat score, effect size.
- * Interpret the result as
 - a. Effect size > 0 : Target group x is more associated with A (bias towards $x-A$).
 - b. Effect size < 0 : Target group y is more associated with A (Bias towards $y-A$).
 - c. Effect size ≈ 0 : No detectable bias

PROGRAM:

```
import numpy as np
from gensim.models import KeyedVectors
import gensim.downloader as api
import matplotlib.pyplot as plt

model = api.load("word2vec-google-news-300")
```


Output:

Computing WEAT Gender-Career Bias

RESULT

WEAT Effect Size: 0.3136

Bias Direction: Career is more associated with

MALE terms.


```

x-gender = ["man", "male", "boy"]
y-gender = ["woman", "female", "girl"]
A-career = ["Career", "business", "salary"]
B-family = ["family", "name", "children"]
x-european = ["adam", "grey", "harry", "josh"]
y-african = ["jamel", "leony", "tyone"]
def association(w, A, B, model):
    return np.mean([model.similarity(w, a)
    for a in A]) - np.mean([model.similarity(y, a, B,
    model) for y in y])
def effect_size(x, y, A, B, model):
    x-assoc = association(x, A, B, model) for x in x
    y-assoc = association(y, A, B, model) for y in y
    mean-diff = np.mean(x-assoc) - np.mean(y-assoc)
    std-dev = np.std(x-assoc + y-assoc)
    return mean-diff / std-dev
West-gender = West-score(x-gender, y-gender, A-career,
    B-family, model)
effect-gender = effect-size(x-gender, y-gender,
    A-career, B-family, model)
print(West-gender)
print(effect-gender)
if effect-gender > 0:
    print("Bias detected: Male terms")
elif effect-gender < 0:
    print("Bias detected: Female terms")
else:
    print("No detectable bias")

```

RESULT:

✓ done 10/2/2020 The program was executed successfully.

Ex: 3

DATE: 10.2.26

Generative Adversarial Networks for Synthetic Image Generation

AIM:

To implement Generative Adversarial Networks for Synthetic Image Generation using the MNIST Dataset.

ALGORITHM:

- * Load and Preprocess Data.

- * Load MNIST Dataset

- * Normalize images to range $[-1, 1]$

- * Shuffle and create batches.

- * Build Generator

- * Input: 100-dim random noise vector

- * Dense layer $\rightarrow$ reshape to $7 \times 7 \times 256$

- * Apply BatchNorm + LeakyReLU

- * Use Conv2DTranspose layers to upsample 28×28

- * Output Activation: tanh

- * Build Discriminator

- * Input: $28 \times 28 \times 1$ image

- * Apply Conv2D + LeakyReLU + Dropout layers

- * Flatten

- * Dense layer with sigmoid $\rightarrow$ output real/fake probability.

- * Define Loss Functions

- * Use Binary Cross Entropy

- * Discriminator:

- * Real images $\rightarrow$ label 1

- * Fake images $\rightarrow$ label 0

- * Generator

- * Fake images $\rightarrow$ label 1

Layer (type)	Output Shape	Params #
dense (Dense)	(None, 12544)	1254400
batch normalizer	(None, 12544)	50172
leaky_relu	(None, 12544)	0
reshape	(None, 7, 7, 256)	0
Conv2D Transpose	(None, 7, 7, 128)	819200
Batch Normalization	(None, 7, 7, 128)	512
Leaky-Relu1	(None, 7, 7, 128)	0
Conv-2D Transpose1	(None, 4, 4, 64)	204800
Batch Normalization2	(None, 4, 4, 64)	256
Leaky-Relu2	(None, 4, 4, 64)	0
Conv2D -transpose2	(None, 28, 28, 1)	1600

Total Params: 2,330,944 (8.89 MB)

Trainable Params: 2,305,472 (8.79 MB)

Not Trainable Params: 25,472 (99.50 KB)

Layer (type)	Output Shape	Params
conv2d (conv2d)	(None, 14, 14, 64)	1664
leaky_re_lu-3	(None, 14, 14, 64)	0
dropout	(None, 14, 14, 64)	0
conv2d	(None, 7, 7, 128)	204,928
leaky_re_lu	(None, 7, 7, 128)	0
dropout	(None, 7, 7, 128)	0
flatten	(None, 6272)	0
dense	(None, 1)	6273

Total params: 212,865 (831.50 KB)

Trainable params: 212,865 (831.50 KB)

Non Trainable params: 0 (0.00 KB)

* Define Optimizers

* Use Adam Optimizer for both networks

* Training Step

* Generate random noise

* Produce fake images using Generator

* Compute discriminator loss on real & fake images

* Compute generator loss.

* Calculate gradients

* Update weights

* Training Loop

* Repeat for all batches across epochs

* Periodically generate and display sample images

* Visualization

* Compare generated images with original MNIST images.

CODE%

import tensorflow as tf
~~from tensorflow.keras~~ import layers

import numpy as np

import matplotlib.pyplot as plt.

~~x_train, y_train, x_test, y_test~~ = tf.keras.datasets.mnist.load_data()

x_train = (x_train.astype('float32') - 127.5) / 127.5

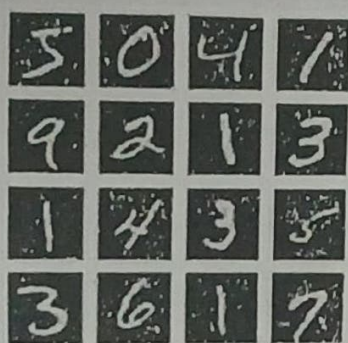
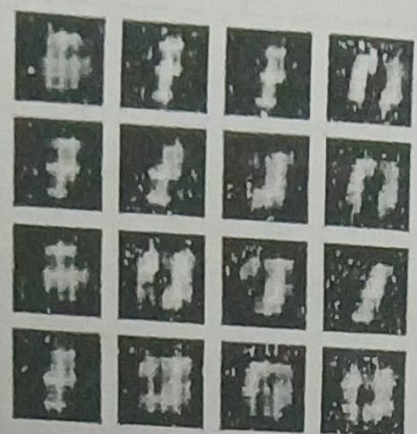
x_train = np.expand_dims(x_train, axis=-1)

BUFFER_SIZE = 60000

BATCH_SIZE = 256

dataset = tf.data.Dataset.from_tensor_slices(x_train)

dataset = dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE, drop_remainder=True)




```

def build_generator():
    model = tf.keras.Sequential()
    layers = Dense(7 * 7 * 256, use_bias=False,
                    input_shape=(100,)),
    layers = BatchNormalization(),
    layers = LeakyReLU(),
    layers = Reshape((1, 7, 256)),
    layers = BatchNormalization(),
    layers = LeakyReLU(),
    layers = Conv2DTranspose(16(5, 5),
    strides = (2, 2), padding = 'same', use_bias=False,
    activation = 'tanh'))

```

return model

generator = build_generator()

generator.summary()

```

def build_discriminator():
    model = tf.keras.Sequential()
    layer = Conv2D(64, (5, 5), strides = (2, 2),
    padding = 'same', input_shape = (28, 28, 3),
    layers = LeakyReLU(), layers = Dropout(0.3),
    layers = Flatten(),
    layers = Dense(1, activation = "sigmoid")
    return model

```

discriminator = build_discriminator()

discriminator.summary()

cross-entropy = tf.keras.losses.BinaryCrossEntropy()

```

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(1 + 0.0001 - leak(real_output), real_output)
    fake_loss = cross_entropy(fake_output, fake_output)

```

return real_loss + fake_loss

```

def generator_loss(fake_output):

```

return cross_entropy(tf.ones_like(fake_output), fake_output)

def train_step(images):

noise = tf.random.normal([batch_size, 100])

with tf.GradientTape():

fake_images = generator(noise, training=TRUE)

real_output = discriminator(fake_images,
training=TRUE)

gen_gradients = gen_tape.gradient(gen_loss,

generator.trainable_variables)

generator_optimizer.apply_gradients

zip(gen_gradients, generator.trainable)

return gen_loss, disc_loss

EPOCHS = 10

seed = tf.random.normal([1, 100])

for epoch in range(1, EPOCHS + 1):

for image_batch in dataset.take(100):

gen_loss = train_step(image_batch)

generate_images(generator, epoch, seed)

print(f"Epoch {epoch} | Gen loss: {gen_loss} |
Disc loss: {disc_loss}")

RESULTS

Thus, to implement Generative Adversarial
Networks (GANs) for Synthetic Image
Generation using MNIST Dataset is
executed successfully.

Ex No: 4

DATE:

Comparative Analysis of Large Language Models

AIM:

To develop a python program that performs a comparative analysis of large language models by analyzing the text generation performance of OpenAI, Gemini and Hugging Face models.

ALGORITHM:

- * OpenAI, Hugging Face, Transformers, Gemini API.
- * Configure OpenAI Key and Gemini API Key.
- * Write function to send prompt and receive response from OpenAI, Gemini & Hugging Face models.
- * Provide a common text prompt.
- * By sending the same prompt to all three models and collect their responses.
- * Individually use Markdown / HTML formatting to display results clearly.
- * Construct an evaluator.
- * Display the ranked model names in best $\rightarrow$ worst order.


```

import openai
from transformers import AutoModelForCausalLM,
AutoTokenizer, pipeline
from IPython.display import Markdown, display
import google.generativeai as genai
os.environ["GOOGLE_API_KEY"] = input("Enter your
Google API Key:")
genai_client = genai.Client(api_key=os.getenv(
"GOOGLE_API_KEY"))
print("In loading Hugging Face GPT-2 model...")
hf_generator = pipeline("text-generation", model="gpt2")
def query_genai(prompt):
    response = genai_client.models.generate(
        content(model="gemini-2.0-flash",
        contents=prompt)
    )
    return response.text
def query_huggingface(prompt):
    response = hf_generator(
        prompt, max_length=60,
        num_return_sequences=1)
    return response[0]["generated_text"]

```

Prompt = "write a haiku about AI and nature"

print("In Generating responses...")

OUTPUT:

loading Model

loading HuggingFace GPT-2 model.

Generating responses...

Gemini Response:

Silicon forests breathe,

Algorithms watch the wind,

Nature Codes in light

HuggingFace Response:

AI whispers soft

through circuits and ancient trees,

learning from the wind.

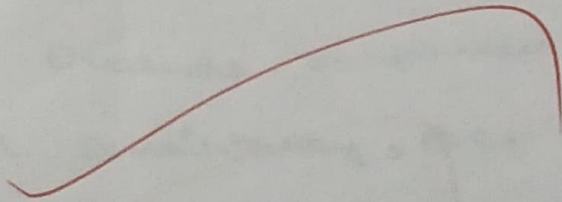
Evaluation:

1st: Gemini

2nd: HuggingFace


```
gmini-output = query-gmini(prompt)
hf-output = query-huggingface(prompt)
print (" Model Outputs \n")
print (" Gemini Response \n")
print (" HuggingFace Response: \n")
print (hf-output)

evaluation = query-gmini (evaluation-prompt)
print (evaluation)
```



RESULTS

The program successfully generated ~~response~~ using Gemini & Huggingface API-2 models & evaluated from using Gemini. Gemini produced most relevant & creative output.

EX NO: 5

DATE:

Large Language Models

AIM:

To analyse how zero shot, few shot and chain of thought prompting techniques affect the reasoning capability of LLMs and identify patterns of logical fallacies and hallucinations in generated responses.

ALGORITHM:

- * Define reasoning dataset.
- * Define Prompting strategies.
- * Generate responses from LLM.
- * Evaluate responses for ~~core~~ correctness, logical consistency, etc.
- * Detect logical fallacies.
- * Calculate metrics.

CODE:

```
import random
from collections import defaultdict

class Prompting Mode:
    zero-shot = "zero-shot"
    few-shot = "few-shot"
    chain-of-thought = "chain-of-thought"
```


class LLMSimulator:

def __init__(self):

self.accuracy_rates = {}

PromptingMode.Zero-Shot = 0.55,

PromptingMode.Few-Shot = 0.70,

PromptingMode.Chain-of-Thought = 0.8,

self.hallucination_rates = {}

PromptingMode.Zero-Shot = 0.30,

PromptingMode.Few-Shot = 0.18,

PromptingMode.Chain-of-Thought = 0.10,

self.fallacy_rates = {}

PromptingMode.Zero-Shot = 0.25,

PromptingMode.Few-Shot = 0.15,

PromptingMode.Chain-of-Thought = 0.08,

def evaluate(self, mode, trials=1000):

results = defaultdict(int)

for _ in range(trials):

if random.random() < self.

accuracy_rates[mode]:

results["correct_reasoning"] += 1

else:

results["incorrect_reasoning"] += 1

return results

Output:

Mode: Zero-shot

Correct-reasoning: 0.55

Incorrect-reasoning: 0.45

Hallucination: 0.30

Logical-fallacy: 0.25

Mode: Few-shot

Correct-reasoning: 0.71

Incorrect-reasoning: 0.15

Hallucination: 0.29

Logical-fallacy: 0.18

Mode: Chain-of-Thought

Correct-reasoning: 0.82

Incorrect-reasoning: 0.08

Hallucination: 0.18

Logical-fallacy: 0.10


```

def run_experiment (trials = 1000):
    simulator = LLMsimulator()
    for mode in [
        PromptingMode.Zero-Shot,
        PromptingMode.Few-Shot,
        PromptingMode.Chain-of-Thought,
    ]
        results = simulator.evaluate(mode,
                                     trials)
        print(f"\n Mode: {mode}")
        for key, value in results.items():
            print(f"{key}: {value} trials: {trials}")

if __name__ == "__main__":
    run_experiment (trials = 5000)

```

RESULT:

Thus, to analyze how zero-shot, Few-shot & Chain-of-Thought prompting techniques affect the reasoning capability of LLMs & identify patterns are executed successfully.

Ex no: 6

Date: 10.8.26

Semantic Similarity Search for Movies using High Dimensional Vector Embeddings with FAISS

AIM:

To implement a semantic similarity search system that finds similar movies using vector embeddings and FAISS (Facebook AI Similarity Search).

ALGORITHM:

- * Import req libraries such as pandas, numpy and transformers.
- * Load the dataset (Amazon Products Movie dataset)
- * Select the relevant text column.
- * Load a pretrained embedding model.
- * Normalize embeddings for better similarity comparison.
- * Build a FAISS index using the embedding
- * Define a similarity search function that:
 - * Converts query text into embedding
 - * Searches nearest vectors in FAISS index
- * Perform similarity search using sample.
- * Display the most similar result.

CODE:

```
!pip install gais -cpu transformers torch pandas
from google.colab import files
uploaded = files.upload()

import pandas as pd
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
print("Dataset loaded Successfully")
print(df.head())

Text_Column = df.columns[0]
item-text = df[Text_Column].astype(str).to_list()

print("Total items:", len(item-text))
print("Sample item:", item-text[0])

from transformers import AutoTokenizer,
AutoModel

import torch
import numpy as np
import gais

device = "cuda" if torch.cuda.is_available()
else "cpu"

model_name = "sentence-transformers/all-MiniLM
L6-v2"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModel.from_pretrained(model_name)
.to(device)

model.eval()
```



```

def encode_texts (texts, batch_size=282):
    all_embeddings = []
    with torch.no_grad():
        for i in range(0, len(texts), batch_size):
            batch = texts[i:i+batch_size]
            tokens = tokenizer(
                batch, padding=True,
                truncation=True,
                return_tensors="pt").
                to(device)
            outputs = model (**tokens)
            all_embeddings.append(outputs)
        return torch.cat(all_embeddings)

item_embeddings = encode_texts (item_texts)
faiss = normalize_L2 (item_embeddings)
print (item_embeddings.shape)

embedding_dim = item_embeddings.shape[1]
index = faiss.IndexFlatL2 (embedding_dim)
index.add (item_embeddings)
print (index.ntotal)

```

```

def get_similar_items (item_id, k=5):
    query_vector = item_embeddings [item_id:
                                     item_id+1]
    scores, indices = index.search (query_vector, k)
    return scores[0:], indices[0:]

```


OUTPUT:

Dataset loaded successfully

	category - name
0	Beading & Sewing Making
1	Fabric Decorating
2	Knitting & Crochet Supplies
3	Printmaking Supplies
4	Scrapbooking & Stamping

Total items: 248
Sample item: 1

{ "model_id": "boc0a89d1bb5e41788", "version_major": 1, "version_minor": 0 }

key

Status

embeddings.position-ids Unexpected

Embeddings shape: (248, 384)

FAISS index built with 248 items

QUERY ITEMS:

SIMILAR ITEMS:

- 1 (score = 1.000)
- 2 (score = 0.805)
- 3 (score = 0.791)
- 4 (score = 0.769)
- 5 (score = 0.658)

query-id = 0

scores, ids = get-similar-items(query-id, k=5)

print (item-texts [query-id])

for score, id in zip (scores, ids):

print (f"→ {item-texts[id]} (score
= {score * .383})")

RESULT:

Thus, the semantic similarity search system was successfully implemented using FAISS and sentence embeddings.

Ex no: 7

DATE: 17. 8. 26

LLM to build RAG

AIM:

To Integrate a Vector Database with an LLM to build Retrieval Augmented Generation system that answers questions based on external PDF Documents.

ALGORITHM:

- * Start the program.
- * Load the PDF document.
- * Extract text from the PDF
- * Split the text into smaller chunks.
- * Convert text chunks into embeddings using an embedding model.
- * Store embeddings in a vector database.
- * Accept the user query.
- * Convert query into embeddings.
- * Retrieve relevant chunks from the vector database.
- * Pass retrieved context along with the query to the LLM.
- * Generate the answer using LLM.
- * Display the result
- * Stop the program.

CoDB :

!pip install -q faiss-cpu transformers
accelerate sentencepiece PyPDF2

~~#~~ ----- Imports -----

import faiss

import numpy as np

import torch

~~from~~ from transformers import AutoTokenizer

from PyPDF2 import PdfReader

from google.colab import files

device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

enc_name = "sentence-transformers/all-MiniLM-L6-v2"
enc_tokenizer = AutoTokenizer.from_pretrained(enc_name)
enc_model = AutoModel.from_pretrained(enc_name, to_device)
enc_model.eval()

lm_name = "TinyLlama/TinyLlama-1.1B-Chat-V1.0"

lm_tokenizer = AutoTokenizer.from_pretrained(lm_name)

lm_tokenizer.pad_token = lm_tokenizer.eos_token

lm_model = AutoModelForCausalLM.from_pretrained(

lm_name, torch_dtype=torch.float16

if device == "cuda" else torch.float32,

device_map="auto"

)

lm_model.eval()

uploaded = files.upload()

pdf_path = list(uploaded.keys())[0]


```

def extract-text-from-pdf(path):
    reader = PDFReader(path)
    text = ""
    for page in reader.pages:
        page-text = page.extract-text()
        text += page-text + "\n"
    return text

```

```

def chunk-text:
    words = text.split()
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk-size
        chunks.append(" ".join(
            words[start:end]
        ))
        start = chunk-size + 1
    return chunks

```

```

def generate-ask-answer(question, top-k=5,
    max-new-tokens=256):
    contexts = retrieve-contexts(q, top-k)
    context-text = "\n\n" + join(contexts)
    prompt = ("you are an assistant
        that answers only using
        contexts")
    inputs = llm-tokenizer(prompt,
        return-tokens="pt", truncate
            = True)

```


OUTPUT :

Total - chunks: 9927

This is a friendly reminder the current text generation call has exceeded the model's predefined maximum length.

Answer = You are an assistant that answers questions strictly using the provided context.

Context :

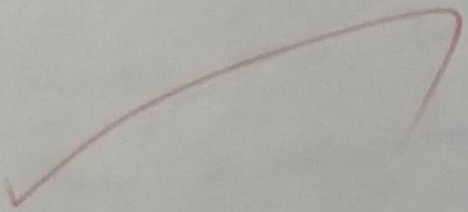
~~Shelly~~
Z Curry Aristocrat A.I.M Cash &
Carry Aristocrat Point - B string
Bridge Consulting & Expec^oa.

Output-text = llm-tokenizer.decode()

return output-text.strip()

question = "What is the main
contribution of this paper?"

answer = generate_rag_answer(question)



RESULT:

Thus the program was executed

~~It~~ successfully.

Ex: 8

DATE: 24.8.26

Deploy a fine tuned
language model in
Hugging face spaces for,
Streamlit for Customer
Support, evaluate using
Bouge & Bleu

AIM:

To deploy a fine tuned language
model, Hugging face spaces.

PROCEDURE:

- * Import necessary libraries
- * Upload the files
- * Load the Bank FAQ dataset
- * Read the dataset & csv files
- * Deploy fine tuned language
- * Use the face space, streamlit
- * By evaluate it by Bouge & Bleu

PROGRAM:

from dates import load_data
from transformer import Tokenizer
import evaluate, matplotlib.pyplot as plt

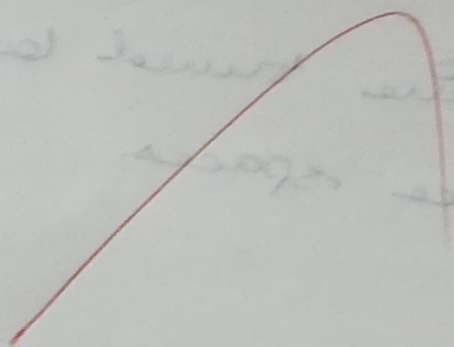
OUTPUT:

BLEU: 0.72

ROUGE₁: 0.80

ROUGE₂: 0.08

ROUGE₃: 0.77




```
data = load_dataset ["Tolui", data_files =  
["train", "train_val"]]
tokenizer = Tokenizer.from_instances(pretrained  
(tokenizer))
```

```
def f(x):
```

```
    i = torch.cat(["question:", x["question"]],  
                  truncate=True)
```

```
    padding = "max_length", max_length=64)
```

```
    i["labels"] = 0 ["input_ids"];
```

```
    return i
```

```
data = data.map(f)
```

```
model = torch.nn.Lstm(100, 100, batch_first=True)  
trainer = GradientDescentTrainer(model, data_loader, dev_loader)
```

```
trainer.train()
```

```
bleu, Rouge = evaluate_loader("bleu")  
evaluate_loader("Rouge")
```

```
for x in data["text"]:
```

```
    out = model.generate()
```

```
    b = bleu.compute()
```

```
    plt.bar(["Blue", Rouge["I"]])
```

```
    plt.show()
```

RESULT:

Thus, the program was executed
successfully.